Instituto Superior de Formación Técnica Nº 151

Carrera: Analista en Sistemas

X Año. Algoritmos y Estructuras de Datos X.

Trabajo Práctico Nº2	**Unidad X2- Python**
Modalidad: Semi -Presencial	**Estratégica Didáctica:** Trabajo individual.
Metodología de Desarrollo: Det. docente	**Metodología de Corrección:** Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	**Fecha Entrega:** A confirmar por el Docente.

INTRODUCCION A PYTHON.

1. Instalación y Entorno

Marco Teórico:

1.1 ¿Que la Consola Interactiva?, que permite y facilita.
1.2 Que es un Repositorio de Paquetes, que repositorios existen.
1.3 Que es un Paquete en Python, como se descarga un Paquete y como se Gestiona.
1.4 ¿Qué es un Entorno Virtual?, para que Sirve, ¿como se gestiona? Dar Ejemplos.

Marco Practico: Realizar en Python

1.5 Lanzar la Consola Interactiva (Python o Python3 en Linux/Mac)
1.6 Imprimir por Consola “Hola Mundo”
1.7 Lanzar el IDLE y repetir el punto 2
1.8 Añadir el Repositorio PIP: python get-pip.py
1.9 Actualizar el PIP: pip install --upgrade pip setuptools (Linux: sudo apt update && sudo apt upgrade python3-pip)
1.10 Probar PIP (list – show - update)

2. Aspectos del Lenguaje.

Marco teórico

2.1 Cuál es el concepto de Variables que maneja Python a diferencia de otros lenguajes.
2.2 ¿Qué tipos de Datos maneja Python?
2.3 Describir Operaciones aritméticas y orden de precedencia.
2.4 Dar ejemplos del Comando IF.

Marco Practico: Realizar en Python

2.1 Crear un entorno virtual nuevo
2.2 Activar un entorno virtual
2.3 Instalar el Paquete Flask
2.4 Desactivar un entorno virtual
2.5 Borrar el entorno virtual

3. Colecciones en el Lenguaje.

Marco Teórico

3.1 Que es una Tupla, como funciona, dar ejemplos.
3.2 Que es una Lista, como funciona, dar ejemplos.
3.3 Que es Diccionario, como funciona, dar ejemplos.

Marco Practico

3.1 Mete los valores del 1 al 100 en una lista.
3.2 Crea una tupla con los meses del año, pide números al usuario, si el numero esta entre 1 y la longitud máxima de la tupla, muestra el contenido de esa posición sino muestra un mensaje de error.
3.3 Pide un numero por teclado y guarda en una lista su tabla de multiplicar hasta el 10. Por ejemplo, si pide el 5 la lista tendrá: 5,10,15,20,25,30,35,40,45,50
3.4 Crea un diccionario donde la clave sea el nombre del usuario y el valor sea el teléfono (no es necesario validar). Tendrás que ir pidiendo contactos hasta el usuario diga que no quiere insertar más. No se podrán meter nombres repetidos.

4. Bucles en el Lenguaje.

Marco teórico

4.1 Describir las sentencias While y For
4.2 Describir y dar ejemplos de Range y Switch
4.3 Que función tiene la sentencia Breack y Continue

Marco Practico

4.1 Escriba un programa que pida dos números enteros y escriba qué números son pares y cuáles impares desde el primero hasta el segundo.
4.2 pide un número positivo al usuario una y otra vez hasta que el usuario lo haga correctamente
4.3 Escriba un programa que pida dos números enteros. El programa pedirá de nuevo el segundo número mientras no sea mayor que el primero. El programa terminará escribiendo los dos números.
4.4
Escriba un programa que pida dos números enteros y escriba la lista de números consecutivos que hay entre ellos, de menor a mayor.

5. Funciones en Python | Recursividad | Funciones Lambda | programación Funcional.

Marco teórico

5.1 Como se Define una función en Python, explicar la forma de manejo de los Argumentos.
5.2 Definir como Python maneja el pasaje x valor y x referencia.

5.3 Definir los Argumentos “Especiales” en Python: *Args y **Kwargs en Python
5.4 Dar ejemplos de *Args y **Kwargs en Python
5.5 Que es recursividad, que tipos de problemas soluciona, sar ejemplos.
5.6 Que es una Función Lambda, que permite o facilita, de ser posible, completamente la diferencia con otros lenguajes

Marco Practico

5.1 Escriba un programa que pida la anchura y altura de un rectángulo y el caracter a utilizar en el dibujo
5.2 Escriba un programa que pida un año y que escriba si es bisiesto o no.
Se recuerda que los años bisiestos son múltiplos de 4, pero los múltiplos de 100 no lo son, aunque los múltiplos de 400 sí. Estos son algunos ejemplos de posibles respuestas: 2012 es bisiesto, 2010 no es bisiesto, 2000 es bisiesto, 1900 no es bisiesto
5.3 Escriba un programa que permita crear una lista de palabras (que puede ser vacía). Para ello, el programa tiene que pedir un número y luego solicitar ese número de palabras para crear la lista. Por último, el programa tiene que escribir la lista.

6. programación Funcional en Python

6.1 Que entiende por programación Funcional
6.2 Que herramientas provee Python para la programación Funcional
6.3 Para que sirve la función Map, ¿que facilita o permite?
6.4 ¿Que utilidad tiene la Función Filter?, como se realizaría en estructurado?
6.5 ¿Que permite la Función Reduce en Python?
6.6 Explicar el Siguiente Código de Ejemplo de programación Funcional

```
from functools import reduce
personas = [
    {'Nombre': 'Alicia', 'Edad': 22, 'Sexo': 'F'},
    {'Nombre': 'Bob', 'Edad': 25, 'Sexo': 'M'},
    {'Nombre': 'Charlie', 'Edad': 33, 'Sexo': 'M'},
    {'Nombre': 'Diana', 'Edad': 15, 'Sexo': 'F'},
    {'Nombre': 'Esteban', 'Edad': 30, 'Sexo': 'M'},
    {'Nombre': 'Federico', 'Edad': 44, 'Sexo': 'M'},
]
hombres = list(filter(lambda x: x['Sexo'] == 'M', personas))
suma_edades = reduce(lambda suma, p: suma + p['Edad'], hombres, 0)
media_edad = suma_edades/(len(hombres))
print(media_edad) # 33.0
```

6.7 De ser Posible comparar con C++

Marco Practico:

6.1 Obtener el cuadrado de todos los elementos en la lista.
6.2 Obtengamos la cantidad de elementos mayores a 5 en una tupla.
6.3 Obtengamos la cantidad de elementos mayores a 5 en una tupla, usando Reduce

7. Excepciones en Python

Marco Teorico

7.1 Que es una Excepción, que significa “Lanza” una Excepción y “Capturarla”
7.2 Complementar con ejemplos de TypeError, KeyError, IndexError, NameError, RuntimeError, ZeroDivisionError
7.3 Que utilidad tiene la palabra Raise.

Marco Practico

7.1 Crear una Función que divia hasta cero, ej: dividir(27,0), verificar: ZeroDivisionError:
7.2 llamar a la función mas_10() con cualquier número, además: add_10("cinco") verificar TypeError:
7.3 Crear una Lista e Iterar mas allá del limite del Index, verídica: IndexError:
7.3 Crear un Diccionario en Python y buscar una clave Inexistente, verificar keyError:

8. Archivos en Python

Marco teórico

8.1 Explicar las Funciones Open() y Close() de Python.
8.2 Explicar los distintos “Modos” de Apertura.
8.3 Definir y dar Ejemplos de seek(), readline() y readlines()

Marco Practico

8.1 Crear un programa que abra un fichero en modo lectura y escritura, si no existe lo creará, y añadir la frase “Estoy aprendiendo Python”
8.2 Crear un programa que abra el fichero editado anteriormente y muestre el estado del fichero, el modo en el que fue abierto, el nombre y la codificación de caracteres del mismo.
8.3 Realizar un programa que realice los ejercicios 1 y 2 utilizando la estructura with.

9. Módulos y Paquetes.

Marco teórico

9.1 Definir y relacionar los Conceptos de Módulos y Paquetes.
9.2 Que función tiene sys.path()
9.3 Por qué usaría “AS” para cambiar o poner alias a paquetes en Python.
9.4 ¿Como maneja Python las Excepciones al Importar Módulos?
9.5 Como maneja Python la inclusión Multiple de Módulos

Marco Practico

9.1: Hacer un paquete simple
9.2: Crear un directorio de aplicaciones
9.3: Scripts de nivel superior

10. Testing en Python.

10.1 ¿Que entiende x Testing?
10.2¿Que diferencia tenemos entre Test Manual y Automático?
10.3¿Que es un Assert en Test Automáticos?
10.4 Que es unittest, como se Implementa en Python?
10.5 Explicar el Siguiente Código:

```
# funciones.py

    def calcula_media(*args):
        return(sum(*args)/len(*args))

# tests.py

from funciones import calcula_media
import unittest
class TestCalculaMedia(unittest.TestCase):
  def test_1(self):
    resultado = calcula_media([10, 10, 10])
    self.assertEqual(resultado, 10)
  def test_2(self):
    resultado = calcula_media([5, 3, 4])
    self.assertEqual(resultado, 4)

if __name__ == '__main__':
  unittest.main()
```

10.6 Realizar una Breve descripcion de las siguientes funciones de UnitTes:

.assertEqual(a, b): Verifica la igualdad de ambos valores.
.assertTrue(x): Verifica que el valor es True.
.assertFalse(x): Verifica que el valor es False.
.assertIs(a, b): Verifica que ambas variables son la misma (ver operador is).

.assertIsNone(x): Verifica que el valor es None.
.assertIn(a, b): Verifica que a pertenece al iterable b (ver operador in).
.assertIsInstance(a, b): Verifica que a es una instancia de b
.assertRaises(x): Verifica que se lanza una excepción.

10.7 Que permite Usando setUp y tearDown en UnitTest?

Marco Practico

10.1 Explicar el Siguiente Código:

```
import unittest
class TestEjemplos(unittest.TestCase):
   def setUp(self):
      print("Entra setUp")
   def tearDown(self):
      print("Entra tearDown")
   def test_1(self):
      print("Test: test_1")
   def test_2(self):
      print("Test: test_2")

python -m unittest -v tests
```

La Cátedra